

Reporte de Configuración Base, Repositorio y Entornos

Sistema de Gestión de Habitaciones y Operaciones (GeHO)

Fase del Proyecto	Configuración Base e Infraestructura	Fecha de Entrega	19 de Mayo de 2026
Responsable (DevOps)	Alejandro Martínez Bernal (No. Control: 22100209)	Equipo de Desarrollo	Jorge Arturo Mata Camacho (Backend) Christhofer Solis Zamarron (DBA) Dana Garcia Martinez (Frontend)

1. Inicialización del Repositorio y Control de Versiones

Se ha creado y configurado el repositorio centralizado del proyecto utilizando Git y GitHub como plataforma de alojamiento. El repositorio servirá como la única fuente de la verdad para el código fuente, scripts de bases de datos y documentación técnica del Sistema GeHO.

1.1. Estructura de Directorios Base

Para mantener un entorno de trabajo limpio y modular, se implementó la siguiente estructura inicial de carpetas, forzando al equipo a mantener el orden arquitectónico:

```
Sistema-GeHO/  
  .github/workflows/ # Scripts de automatización y CI  
  database/  
    mongodb/ # Scripts y esquemas JSON  
    sql-server/ # Scripts DDL/DML relacional  
  docker/ # Archivos docker-compose.yml  
  docs/  
    diagrams/ # Mockups UI y diagramas DB  
    evidence/ # Documentación y reportes
```

```
src/  
  backend/ # Web API C# (Jorge)  
  frontend/ # Aplicación cliente (Dana)
```

2. Estrategia de Ramas (Git Flow)

Se estableció un flujo de trabajo colaborativo profesional para evitar conflictos de código entre los desarrolladores (Frontend, Backend y DBA). La arquitectura de ramas se divide en:

- **main** (Rama de Producción): Contiene el código estable y las versiones finales aprobadas del sistema.
- **develop** (Rama de Integración): Entorno principal de trabajo donde se unen las características antes de pasar a producción.
- **feature/*** (Ramas de Tareas): Cada miembro del equipo debe crear ramas temporales (ej. *feature/login-ui*, *feature/api-checkout*) para trabajar de forma aislada.

3. Políticas de Seguridad y Protección del Entorno

Para blindar la integridad del proyecto, el líder de DevOps aplicó bloqueos nativos en la plataforma de GitHub sobre las ramas críticas (*main* y *develop*):

- **Prohibición de Push Directo:** Ningún miembro del equipo puede subir código directamente a las ramas principales.
- **Pull Requests Obligatorios (Peer Review):** Todo nuevo código debe ser propuesto mediante un Pull Request y requiere al menos 1 aprobación (Approve) de otro miembro del equipo o del administrador.
- **Validación de Integridad:** Se exige que el robot de automatización apruebe la estructura de archivos sin detectar errores antes de habilitar la fusión.

4. Automatización Inicial (CI/CD)

Se configuró un entorno de Integración Continua (CI) mediante GitHub Actions. El script automatizado ejecuta un job de validación (*check-repo*) cada vez que se intenta integrar código nuevo, asegurando que nadie guarde archivos en la raíz del proyecto y se respete la arquitectura definida.

5. Preparación del Entorno de Contenedores

Se generó el entorno estructural que alojará la orquestación de Docker Compose. Este modelo virtualizado permitirá levantar simultáneamente la base de datos SQL Server, la base de datos

MongoDB y la Web API ASP.NET Core sin necesidad de instalaciones locales en los equipos de los desarrolladores.